

Contents

Introduction	2
Terminology	2
Methodology	2
Sorting algorithms to compare against	2
Notable metrics	3
Parameters	3
Data generation	3
Computer Specifications	4
Data Collection Methods	6
Algorithm Explanation	7
Overview of Binary Insertion Sort	7
Merging	8
Timsort	12
Java implementation	17
Code structure	17
Usage	17
Timsort implemenation	17
Key characteristics	19
Time complexity analysis	19
Best-case scenario	19
Average-case scenario	19
Worst-case scenario	21
Space complexity	21
Comparison with other sorting algorithms	21
Performance results and comparison with other algorithms	22
Runtime	22
Comparisons	23
Swaps	24
Copies (excluding swaps)	25
Copies (including swaps)	26
Use Cases & Real-World Examples	27
Conclusion and Limitations	27

Introduction

Timsort is a reliable hybrid sorting algorithm that uses the combination of the Merge Sort and the Binary Insertion Sort. It is an adaptable sorting algorithm, meaning it exploits pre-existing orders sequences (runs) within the input dataset to minimize comparisons and operational overhead, which is different from the $O(n \log_2 n)$ comparison sorts that treat data partially ordered identical and random.

Timsort was designed by Tim Peter in 2002 originally for the Python library, and it was surprisingly successful in handling real life work data and led to global use till these days. Furthermore, Timsort today serves as the default sorting algorithm for object arrays in the Java language (`Arrays.sort`)¹.

This research report evaluates the theoretical mechanics and practical test of our Java execution of Timsort.

Terminology

There are a couple of terminologies that I will refer to:

- run: a run refers to a subarray that's already sorted
- invariant: a particular trait, condition, or constraint that must hold for all cases. For instance, a sorted array in ascending order has the invariant that $a[i + 1] \geq a[i] \forall i \in [0, n - 2]$ where n is the size of the array

Methodology

We will mostly be empirically determining Tim sort's performance through sample trials and data analysis. We will then see whether the data matches with the theoretical time complexity of the algorithm.

We will then compare it against other sorting algorithms to see why it is one of the most implemented sorting algorithm.

Sorting algorithms to compare against

We will compare it against the following sorting algorithms:

- Bubble Sort
- Insertion Sort
- Selection Sort
- Merge Sort
- Timsort
- Java's Sort

¹The source code for JDK11 for `Arrays.sort` is linked here

The reason why we included Merge Sort is because Timsort builds off of Merge Sort. Moreover, our “control” will be Java’s sorting algorithm. Also, do note that our Insertion Sort implementation is Insertion Sort with binary search, that is, Binary Insertion Sort, because, we have to utilize it for Timsort.

Notable metrics

For every trial run, we will keep track of the following metrics:

- runtime
- number of comparisons
- number of swaps
- number of copies (excluding swaps)
- number of copies (including swaps)

Swap consists of 3 copies, and since some algorithms don’t really use conventional swapping, we believe it’s a better metric to compare in place of the amount of swaps. In the code, the `copies` variable is the amount of copies excluding the amount of swaps.

Parameters

Tim sort has a adjustable parameter, which is the minimum run length. For our particular use case, we will use a minimum run size of 32.

Data generation

The most obvious data set we should use is integers, however, that begs the question, what should be the range of the randomly generated integers?

Suppose that we generate random integers in the range between $[1, k]$ and we generate a list of size n . If $k \gg n$, then we would rarely get duplicates, meanwhile, if $k \ll n$, then we would get many duplicates.

Our parameter will then be the ratio $r = \frac{n}{k}$ where a high value of r would mean more duplicates.

In our case, we selected 2 duplication ratios:

- $r = 10$ for data with high duplication
- $r = 0.1$ for data with low duplication

For each duplication ratios, we will use datasets of the following sizes:

- $n = 5$ for small datasets (for data with high duplication, $k = 1$, meaning that the data will just be an array containing just 1)
- $n = 1000$ for medium sized datasets
- $n = 1000000$ for large datasets

Computer Specifications

To make sure that the run time results aren't too much affected by other external factors, we ran the same algorithms on the same computer.

The most important variable for run time in this case is probably the CPU, and the speed and size of the RAM.

In our case, we used a CPU with the following `lscpu` output:

```
Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
Address sizes:               39 bits physical, 48 bits virtual
Byte Order:                  Little Endian
CPU(s):                      6
On-line CPU(s) list:        0-5
Vendor ID:                   GenuineIntel
Model name:                  Intel(R) Core(TM) i5-9500 CPU @ 3.00GHz
CPU family:                  6
Model:                       158
Thread(s) per core:         1
Core(s) per socket:         6
Socket(s):                   1
Stepping:                    10
Microcode version:          0xfa
CPU(s) scaling MHz:         20%
CPU max MHz:                 4400.0000
CPU min MHz:                 800.0000
BogoMIPS:                    6000.00
Flags:                       fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca
                             cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht
                             tm pbe syscall nx pdpe1gb rdtscp lm constant_tsc art ar
                             ch_perfmon pebs bts rep_good nopl xtopology nonstop_tsc
                             cpuid aperfmperf pni pclmulqdq dtes64 monitor ds_cpl vmx
                             smx est tm2 ssse3 sdbg fma cx16 xtpr pdcm pcid sse4_1 s
                             se4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave a
                             vx f16c rdrand lahf_lm abm 3dnowprefetch cpuid_fault epb
                             pti ssbd ibrs ibpb stibp tpr_shadow flexpriority ept vp
                             id ept_ad fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms i
                             nvpcid mpx rdseed adx smap clflushopt intel_pt xsaveopt
                             xsavec xgetbv1 xsaves dtherm ida arat pln pts hwp hwp_no
                             tify hwp_act_window hwp_epp vnmi md_clear flush_l1d arch
                             _capabilities
```

Virtualization features:

```

Virtualization:          VT-x
Caches (sum of all):
  L1d:                   192 KiB (6 instances)
  L1i:                   192 KiB (6 instances)
  L2:                    1.5 MiB (6 instances)
  L3:                    9 MiB (1 instance)
NUMA:
  NUMA node(s):          1
  NUMA node0 CPU(s):     0-5
Vulnerabilities:
  Gather data sampling:   Mitigation; Microcode
  Ghostwrite:            Not affected
  Indirect target selection: Not affected
  Itlb multihit:         KVM: Mitigation: Split huge pages
  L1tf:                  Mitigation; PTE Inversion; VMX conditional cache flushes
                        , SMT disabled
  Mds:                   Mitigation; Clear CPU buffers; SMT disabled
  Meltdown:              Mitigation; PTI
  Mmio stale data:       Mitigation; Clear CPU buffers; SMT disabled
  Old microcode:         Not affected
  Reg file data sampling: Not affected
  Retbleed:              Mitigation; IBRS
  Spec rstack overflow:   Not affected
  Spec store bypass:     Mitigation; Speculative Store Bypass disabled via prctl
  Spectre v1:            Mitigation; usercopy/swapgs barriers and __user pointer
                        sanitization
  Spectre v2:            Mitigation; IBRS; IBPB conditional; STIBP disabled; RSB
                        filling; PBRSE-eIBRS Not affected; BHI Not affected
  Srbds:                 Mitigation; Microcode
  Tsa:                   Not affected
  Tsx async abort:       Mitigation; TSX disabled
  Vmscape:               Mitigation; IBPB before exit to userspace

```

As for the RAM, the output of `sudo dmidecode --type 17` is as follows:

```

Getting SMBIOS data from sysfs.
SMBIOS 3.1.1 present.

```

Handle 0x000A, DMI type 17, 40 bytes

Memory Device

```

  Array Handle: 0x0009
  Error Information Handle: Not Provided
  Total Width: 64 bits

```

Data Width: 64 bits
Size: 8 GiB
Form Factor: DIMM
Set: None
Locator: DIMM1
Bank Locator: Not Specified
Type: DDR4
Type Detail: Synchronous
Speed: 2666 MT/s
Manufacturer: 80CE000080CE
Serial Number: 343C8B13
Asset Tag: 02193900
Part Number: M378A1K43DB2-CTD
Rank: 1
Configured Memory Speed: 2666 MT/s
Minimum Voltage: Unknown
Maximum Voltage: Unknown
Configured Voltage: 1.2 V

Handle 0x000B, DMI type 17, 40 bytes

Memory Device

Array Handle: 0x0009
Error Information Handle: Not Provided
Total Width: Unknown
Data Width: Unknown
Size: No Module Installed
Form Factor: Unknown
Set: None
Locator: DIMM2
Bank Locator: Not Specified
Type: Unknown
Type Detail: None

The OS might also be a factor. We used a Linux kernel which has a `uname -a` output as follows:

7.2.3-arch1-3

The test would be decently fair when tested one after the other because the CPU and memory utilization for other processes would be similar.

Data Collection Methods

Because of the randomness of the dataset, and the randomness of OS scheduling, it is necessary to run the code multiple time and record the average of all notable metrics alongside with its standard deviation.

We will run perform each run 100 times to ensure a good sample size.

Algorithm Explanation

Overview of Binary Insertion Sort

Since Timsort relies on Binary Insertion Sort to extend runs, we must understand how Binary Insertion Sort works.

Binary Insertion Sort builds off of a very simple idea:

- suppose that we have a barrier separating the left side of the array from the right side of the array, which is at index i let's say
- we say that from index 0 to index i , the array is sorted. Even if the array isn't sorted at all, when $i = 0$, it is trivially sorted
- we then try to move right by increasing i by one and then fix the invariant that from index 0 to index i , the list must be sorted
- this can be done by taking the element at index i and then inserting it in the right place such that the range from index $[0, i]$ is sorted
- to find where to insert, we use Binary Search, which takes $O(\log_2(i + 1))$ in this case, and then we insert by first storing the value at i in a temporary variable, then we right shift from the insertion index, then we place that value at the insertion index.
- we then continue this process until we reach the end of the array.

When doing Timsort, for a small sized array, we would be using Binary Insertion Sort.

So keep note that for Binary Insertion Sort, the invariant is that the range $[0, i]$ must be sorted.

Let's have an example. Suppose we have the array `[4, 2, 3, 5, 1]` and `minRun = 32`. In this case, the run can't really reach 32 because the array is of sized 5, the algorithm would do Binary Insertion Sort.

So we first have a pointer pointing at index 0, but the invariant is trivially met, so we move on to the next $i = 1$. Now, the invariant isn't met, however, given that the invariant was met at $i = 0$, we can easily make the invariant hold for $i = 1$ by insertion. In this case, we insert it in the front as such:

```
i = 1
|
v
[2, 4, 3, 5, 1]
```

Now the invariant is met, we move to the next one:

```
i = 2
|
v
[2, 4, 3, 5, 1]
```

The invariant isn't met again, so we perform binary search from 0 to $i = 2$, which will give the insertion index. In our case, we can see that we must insert it in between 2 and 4.

```
      i = 2
      |
      v
[2, 3, 4, 5, 1]
```

The invariant is now met, so we move to the next index

```
      i = 3
      |
      v
[2, 3, 4, 5, 1]
```

The invariant is also met, so we move to the next index.

```
      i = 4
      |
      v
[2, 3, 4, 5, 1]
```

Now we must fix the invariant by finding the point of insertion through binary search, then inserting it. At the end, binary search will give you an index of insertion at $i = 0$.

```
      i = 4
      |
      v
[1, 2, 3, 4, 5]
```

And now you're done.

Merging

So Timsort also builds off of the merging operation from Merge Sort.

The merging operation takes in 2 sorted arrays, and merge them into one array sorted array.

These two arrays in our case, are neighboring sorted sub-arrays (runs), and when we merge them, we will form another a bigger run.

The idea behind merge-based sorting algorithms is that once our array only have one run left, which was constructed through repeated merging, we will have a sorted array.

We will go through an example. Suppose that we have 2 sorted array as such:

`a_1 = [1, 3, 7]`

And:

```
a_2 = [2, 4, 5]
```

Suppose that we want to merge these two sorted arrays into one array, the array would be of known value, which is `a_1.length + a_2.length`. To do so, we initialize 2 pointers at the start of the array.

```
a_1 = [1, 3, 7]
```

```
  ^  
  |  
i_1 = 0
```

and

```
a_2 = [2, 4, 5]
```

```
  ^  
  |  
i_2 = 0
```

While our final array will empty with its own pointer too:

```
a = [null, null, null, null, null, null]
```

```
  ^  
  |  
i = 0
```

We then compare `a_1[i_1]` with `a_2[i_2]`. Which ever one is smaller, we insert it into our new array `a` and index `i`, and then we increment `i++` and also increment the pointer of the array whose value was smaller.

For our particular example, we see that `a_1[0] < a_2[0]`, as such, we insert `a_1[0]` into our new array at index `i = 0`, and then increment both `i` and `i_1` by one.

```
a = [1, null, null, null, null, null]
```

```
  ^  
  |  
i = 1
```

```
a_1 = [1, 3, 7]
```

```
  ^  
  |  
i_1 = 2
```

```
a_2 = [2, 4, 5]
```

```
  ^  
  |  
i_2 = 0
```

Now we repeat this process until we reach the end of one of the array.

```
a = [1, 2, null, null, null, null]
      ^
      |
    i = 2
```

```
a_1 = [1, 3, 7]
      ^
      |
    i_1 = 2
```

```
a_2 = [2, 4, 5]
      ^
      |
    i_2 = 0
```

Then:

```
a = [1, 2, 3, null, null, null]
      ^
      |
    i = 3
```

```
a_1 = [1, 3, 7]
      ^
      |
    i_1 = 2
```

```
a_2 = [2, 4, 5]
      ^
      |
    i_2 = 1
```

Next:

```
a = [1, 2, 3, 4, null, null]
      ^
      |
    i = 4
```

```

a_1 = [1, 3, 7]
      ^
      |
      i_1 = 2

```

```

a_2 = [2, 4, 5]
      ^
      |
      i_2 = 2

```

Next:

```

a = [1, 2, 3, 4, 5, null]
      ^
      |
      i = 5

```

```

a_1 = [1, 3, 7]
      ^
      |
      i_1 = 2

```

```

a_2 = [2, 4, 5]
      ^
      |
      i_2 = 3

```

Notice that a_2 has reached the end of the array. If we were to try to index a_2 at $i_2 = 3$, we would get an error, as such, we should detect it and terminate out of our loop.

After one of the array has reached the end, we can simply take the “tail” of the array whose index hasn’t reached the end and append it into our new array **a**. In this case, we just append **a_1[2]** to the end of the array.

```

a = [1, 2, 3, 4, 5, 7]
      ^
      |
      i = 6

```

```

a_1 = [1, 3, 7]
      ^
      |
      i_1 = 3

```

```

a_2 = [2, 4, 5]
      ^
      |
      i_2 = 3

```

Finally, we terminate and now, we have a sorted array **a**.

Notice that every loop, we must traverse through all of **a_1** and **a_2**, as such, it would take $O(a_1.length + a_2.length)$ time.

Timsort

Timsort revolves around those 2 operations, although, there are additional logic to decide when it should perform Insertion Sort on the sub-arrays, and when it should perform merging.

In a higher-level overview, Timsort aims to find natural runs, and if the run is smaller than **minRun**, it aims to extend the run to length **minRun** using Insertion Sort. The reason why Insertion Sort is used, I presume, is because when extending the run, Insertion Sort can already take advantage of the already sorted run to the left of it, making it do less work. Afterwards, it will then merge those runs such that the run stack maintains an invariant.

Run finding

Let's have an example. Suppose that we choose that **minRun** = 3, and we have the following array:

```
a = [1, 2, 3, 4, 1, 2, 1, 2]
```

The following will be considered a run:

```

a = [1, 2, 3, 4, 1, 2, 1, 2]
    [1, 2, 3, 4]
      ^
      |
      r_1

```

And we won't extend it to **minRun** because it's already bigger than **minRun** = 3. So now we search for the next run.

```
a = [1, 2, 3, 4, 1, 2, 1, 2]
```

```

[1, 2, 3, 4] [1, 2]
  ^         ^
  |         |
r_1        r_2

```

Now we see that `r_2.length < minRun` for `minRun = 3` and `r_2.length = 2`. As such, we need to extend it to `r_3` as such:

```

a = [1, 2, 3, 4, 1, 2, 1, 2]
    [1, 2, 3, 4] [1, 2, 1]
      ^         ^
      |         |
    r_1        r_2

```

However, it isn't sorted, which is where we use Binary Insertion Sort to sort it.

```

a = [1, 2, 3, 4, 1, 2, 1, 2]
    [1, 2, 3, 4] [1, 1, 2]
      ^         ^
      |         |
    r_1        r_2

```

Now we're left with the final run of length 1. Even though we would like to extend it to `minRun = 3`, we need to detect that we're at the boundary of the array, as such, we can only make a run of length 1.

```

a = [1, 2, 3, 4, 1, 2, 1, 2]
    [1, 2, 3, 4] [1, 1, 2] [2]
      ^         ^         ^
      |         |         |
    r_1        r_2        r_3

```

Merging phase

The next step in Timsort is the merging phase. Although Timsort's merging phase can run as we find runs on the go, we think it's conceptually easier to split the two phases, even at the cost of efficiency.

Let's suppose we store the runs in an array. We can either store the content of the arrays themselves, or just the metadata which includes the starting index of the of the run, and how long the run is. In implementation it's more efficient to store just the metadata rather than the sub-arrays themselves.

Going from the earlier example, let's say that we have an array that stores the runs that we found, and that we extended, call it `r`.

```
r = [r_1, r_2, r_3]
```

What we now need to do now is pop each one of them from the front sequentially from the start to the end, and then added to something called a `runStack`. To re-iterate, we actually do not need this auxiliary array `r`,

and we can just pop it in to `runStack` as we find them, but, we believe it's better, conceptually, to split it into two steps. I will abbreviate `runStack` to `rs` to not clutter things up.

So, this `runStack` isn't an ordinary stack, it is stack that must preserve 2 invariants as follows. If we push onto the stack from the end, where n is the index of the end of the stack, then:

The first invariant states:

`rs[n].length + rs[n - 1].length < rs[n - 2].length`

And, the second invariant states:

`rs[n].length < rs[n - 1].length`

I will abbreviate `rs[n].length` to `rs1[n]`.

When $n - 2$ or $n - 1$ is out of bound, then we simply state that the invariants are followed trivially, such as for edge cases when the `rs.length <= 1`. Meanwhile, when `rs.length == 2`, then we only need the second invariant to follow.

So, we every time we pop an element out of `r` (from the start), we push it onto `rs`, and then repair the invariant.

If `rs` already has the invariants, when we push another run onto `rs` (which may break the invariant), then in order to preserve the invariant, we can follow the following procedure:

Let's say that we've pushed a new run into `rs`. And we can assume that `rs` follows the invariants from index 0 to index $n - 1$ (inclusive). There are a couple of cases when the invariant isn't met. The index of the diagrams below goes from 0 to n from the bottom, upwards.

A stack with the invariants met looks like this:

```
#####          ^ n
#####          |
#####          | 0
```

The invariant is met because it can be verified that , `rs1[n] + rs1[n - 1] < rs1[n - 2]`, and `rs1[n] < rs1[n - 1]`.

Now, there are a few cases that might happen when we add a new run:

1. first invariant met and second invariant met
2. first invariant not met and second invariant met
3. first invariant met and second invariant not met
4. first invariant not met and second invariant not met

For the first case, it is simple because all of our invariants are met and we can continue. For the diagram above, that would happen if we discovered a run and append it as such:

```
##
####
#####
#####
```

The second case may happen when we append something like this:

```
####
####
#####
#####
```

Notice that the first invariant isn't met, but the second invariant is met. In such a case, we would want to merge `rs[n - 1]` with the smaller of `rs1[n]` or `rs1[n - 2]`. Notice also that when we perform the merge, it would not guarantee that the invariant is automatically repaired, however, it would still guarantee that the bottom invariants from `n - 2` downwards (after we perform the merge) must still be met, because we haven't merged them in. As such, we can repeatedly go through our routine until we find that the invariants are met for the top 3, which would already mean that the invariants are already preserved from the bottom down. A more rigorous proof of this might be necessary, however, we will just assume that it can be experimentally verified, and that it is correct.

In our case, we will merge `rs[n - 1]` with `rs[n]` because it's smaller.

```
#####
#####
#####
```

Notice that the invariants actually aren't met, in fact, we're now in case 4, because, not only is the first invariant not met, the second invariant isn't met either. In such a case, we see that the first invariant isn't met, so we don't need to worry too much about the second invariant, as such, we would perform the same procedure in case 2. In our case, we still merge `rs[n]` with `rs[n - 1]` (note that `n` gets updated behind the scenes when we merge, and will always refer to the index of the top of the stack)

```
#####
#####
```

Now that we have `rs.length == 2`, we say that the first invariant is already automatically met, as such, we only care about the second invariant. When the second invariant is violated, but the first invariant isn't violated (case 3), we merge `rs[n]` with `rs[n - 1]`.

```
#####
```

When `rs.length <= 1`, we say that the stack invariants are automatically met, and we can then continue onto the pushing of the next element off of `r`, and we repeat until `r` is exhausted. Notice that experimentally from the example above, whenever we perform the merge, we tend to merge arrays of similar sizes, which is another property which makes Timsort behave in an $O(n \log_2 n)$ manner. Essentially, when we merge arrays that are disproportionate in size, the algorithm would perform in an somewhat quadratic manner, which can

be seen in Insertion Sort, because, Insertion Sort can be thought of as a repeated merging of subarrays as such:

- assume that the subarray from $[0, i - 1]$ is sorted
- the subarray from $[i, i]$ is trivially sorted
- merge those two sub arrays to form a new subarray from $[0, i]$
- increment i by one and repeat this process

Because we're Insertion Sort can be thought of as the worst case scenario for merge-based algorithms, it gives a stronger justification that to merge things efficiently, one must merge subarrays of similar sizes. In our implementation (and in Java's implementation), the process of fixing the stack invariants is called the `mergeCollapse` operation.

After we've exhausted the array of runs we've discovered, we essentially have a run stack `rs` that follows the invariant, which may perhaps look something like this:

```
#####
#####
#####
```

The next step is to now merge it into one continuous run. However, notice that the length of the runs in the stack is increasing when moving from `n` to `0`, and furthermore, it is usually the case that it is "Fibonacci-like" when merging, that is, $\text{sum}(\text{rsl}[i:n]) \approx \text{rsl}[i]$. This can be empirically shown, however, we don't have any theoretical understand why this might happen, only that because $\text{rsl}[i + 2] + \text{rsl}[i + 1] < \text{rsl}[i]$, and the invariant is preserved upwards, the $\text{rsl}[i + 3]$ term and onwards are becoming smaller and smaller compared to the region around $\text{rsl}[i]$, making their sum $\text{rsl}[i:n]$ approximately $\text{rsl}[i]$. This makes it the most optimal for merging, where merging is performed by merging the top 2 runs every time, in the case above:

```
#####
#####
#####
```

Which gets merged into:

```
#####
#####
```

Which then gets merged into:

```
#####
```

Once we only have one run left in the stack, then we can say that the array is sorted.

Java implementation

Code structure

For the code structure, we have a few notable classes:

1. **SortMetrics**: **SortMetrics** essentially implements all of the utility functions used by all of the different sorting algorithms such as swapping, comparisons, right shifts, merges
2. **Sort**: **Sort** is an abstract base class that all sorting algorithm use as a template, so that we can in order to make it easier. The **Sort** template requires a **.sort** method, which will be inherited by all sorting algorithms to be implemented.
3. **TimSort**: the **TimSort** class (and all of the other sorting algorithms) inherits from the **Sort** base class and needs to implement their own sorting machinery. The sorting algorithms will use utility functions from the **SortMetrics** instance in order to log the necessary metrics of interest (such as the number of swaps, copies, comparisons, etc).

Usage

In side of **Main.java** we can use our algorithm as such:

```
class Main {
    public static void main(String[] args) {
        SortMetrics<Integer> sortMetrics = new SortMetrics<Integer>(Integer::compare);
        Integer arr[] = { 3, 4, 5, 17, 100, 1, 222, 2, 2, 2 };
        sortMetrics.runSort(arr, new TimSort<Integer>(sortMetrics, 32, new Integer[arr.length]));
        System.out.println(sortMetrics.getResultsAsJson());
        sortMetrics.writeToFile("saved_output/saved.json");
        sortMetrics.resetMetrics();
    }
}
```

Timsort implemenation

First we will give an explanation of each step using plain English, then I will give snippets of the Java implementation related to the specific step described.

1. **Run Identification and Run Extension**: The algorithm scans the array to search for subarrays that are already sorted. If a descending run is detected, it will quickly reverse it to prevent the worst case of Insertion Sort from executing. In our implementation, this is done by the **findRun** method, which returns the start of the run, and the length of the run. If the algorithm finds that the natural run is shorter than the **minRun**. Timsort will extend it by pulling in adjacent elements using Binary Insertion Sort until the runs reaches the **minRun** threshold.

```
InsertionSort<T> insertionSort = new InsertionSort<T>(sortMetrics);
while (runPtr <= arr.length - 1) {
```

```

Run run = findRun(arr, runPtr);
int endIndex;
// second condition is to check whether the subarray is at the boundary of the array
if (run.length() < minRun && (run.startIndex() + run.length()) < arr.length) {
    endIndex = Math.min(run.startIndex() + minRun - 1, arr.length - 1);
    if (!run.isAscending()) {
        sortMetrics.toggleSign();
    }
    insertionSort.sortSubarray(arr, run.startIndex(), endIndex);
    if (!run.isAscending()) {
        sortMetrics.toggleSign();
    }
    run = new Run(run.startIndex(), endIndex - run.startIndex() + 1, run.isAscending());
} else {
    endIndex = runPtr + run.length() - 1;
}
if (!run.isAscending()) {
    sortMetrics.reverse(arr, runPtr, endIndex);
    run = new Run(run.startIndex(), run.length(), true);
}
runPtr = endIndex + 1;
discoveredRunPtr++;
discoveredRun[discoveredRunPtr] = run;
}

```

2. Merge collapse: As runs are identified and measured, they are then pushed onto a runs stack. In our case, in order to make debugging easier, we decided to decouple the runs finding and extension stage, from the invariant preserving stack stage, by storing each run inside of a `discoveredRun` array first, which will then be looped through to be pushed to the actual `runStack`. To help maintain the optimal performance, merging is triggered dynamically in order for the `runStack` to maintain two invariants: $s[n - 2] > s[n - 1] + s[n]$ and $s[n - 1] > s[n]$. Maintaining this invariant has the property that similar sized subarrays will be merged, making it the most optimal $O(n \log_2 n)$ stack collapsing.

```

for (int i = 0; i <= discoveredRunPtr; i++) {
    runStackPtr++;
    runStack[runStackPtr] = discoveredRun[i];
    mergeCollapse(arr);
}

```

3. Merge Force Collapse: the `mergeForceCollapse` stage of the Timsort algorithm essentially takes a `runStack` that's is guaranteed to follow the 2 invariants and will merge everything in the stack from the top down until we have just one run left, which will be our final sorted array.

```

mergeForceCollapse(arr);

```

Key characteristics

- **Stability:** Timsort is stable, it makes sure that the relative order of equal elements remains unchanged during the sorting process. For sorting objects by multiple fields this is extremely important.
- **Adaptability:** The way that the algorithm adapts to the data exiting structure is so nicely executed. If the segment of the array is already sorted (ascending or descending), it will capitalize on this order to skip redundant comparisons.
- **Comparison based:** Timsort can sort items as long as the object as a notion of an order, that is, for a type T , as long as there is a notion that $t_1 > t_2$ for $t_1, t_2 \in T$. This is different from something like Radix Sort or Counting Sort which requires the array to be an array of integers.

Time complexity analysis

We won't fully derive the time complexity for Timsort, and will refer to external authoritative derivations when necessary. Moreover, we will assume that the time complexity of the other algorithms needs no further elaboration.

Best-case scenario

The best-case scenario for Timsort is exactly the same as the best case as Insertion Sort, because, in the case of Timsort, we would find that we only have one run, and even though there are a lot of conditionals afterwards, it wouldn't go into any of them and would just return. In such a case, since `findRun` is $O(n)$, the best case scenario is $O(n)$.

Average-case scenario

For random data, the average case behaves similarly to Merge Sort, where we perform insertion sort once we've reached a sub array of size `minRun`, because, for random data, it is usually the case that we don't have any natural runs that's bigger than `minRun` (unless your duplication ratio is quite high).

A non-rigorous back-of-the-envelope analysis can be done to find the average case running time of Timsort.

Essentially, if our minimum run length is r_m , then we split the array until we reach a sub array of length r_m .

For convenience, let's assume that r_m is a power of 2 and the size of the array n is also a power of 2.

If d is the depth of the merge tree, typically:

$$n = 2^d$$

However, in this case, we factor r_m so that:

$$n = r_m \frac{2^d}{r_m}$$

Meaning that:

$$n = r_m 2^{d - \log_2(r_m)}$$

So, we stop splitting at depth $d_n = d - \log_2(r_m)$

$$d_n = d - \log_2(r_m) = \log_2\left(\frac{n}{r_m}\right)$$

At the bottom, for $\frac{n}{r_m}$ subarrays of size r_m , we need to do Binary Insertion Sort which is of sized r_m , meaning that it takes: Cr_m^2 time where C is a constant factor for some operation. Since we have to do it for $\frac{n}{r_m}$ subarrays, we get that to sort the base of the tree we require:

$$Cr_m^2 \times \frac{n}{r_m} = Cnr_m$$

time.

Merging from that depth level d_n to the top would then take:

$$Cnd_n = Cn \log_2\left(\frac{n}{r_m}\right)$$

because, at every level, we require n operations for merging each subarrays.

As such, in total, we would need:

$$Cn \log_2\left(\frac{n}{r_m}\right) + Cnr_m$$

$$Cn \log_2(n) - Cn \log_2(r_m) + Cnr_m$$

$$Cn \log_2(n) + C(r_m - \log_2(r_m))n$$

time.

So the leading term is $n \log_2(n)$, as such, the algorithm is $O(n \log_2(n))$, as long as r_m is independent of n .

Worst-case scenario

In the case of Insertion Sort, we know that the worst case happens precisely when the array is in descending order, because, it would need to perform $n - 1$ right shifts, and then $n - 2$ right shifts, and then $n - 3$ right shifts. . . which amounts to $O(\frac{n(n-1)}{2})$ (plus binary search, which is $O(\log_2(n))$, which is a lower order term so we ignore it) which is $O(n^2)$.

Timsort isn't naive, in that it has mechanisms to detect descending runs and then reverse them accordingly, preventing the worst case of Insertion Sort from affecting it. This makes analyzing the worst case for Timsort quite difficult. As such we will refer to a paper that analyzed the Worst-case complexity of Timsort by Nicholas Auger, et al. ²

In the abstract, it is stated that for Python's particular implementation of Timsort, it generally runs in:

$$O(n + n \log_2 \rho)$$

where ρ is the number of runs.

Anyway, we can even suppose that there is an upper bound that $\rho \leq n$, because the total amount of runs has to be less than the length of the array itself:

$$O(n + n \log_2 \rho) \leq O(n + n \log_2 n)$$

So:

$$O(n + n \log_2 \rho) \leq O(n \log_2 n)$$

Meaning that in the worst case, it is also $O(n \log_2 n)$ for Python's particular implementation of Timsort.

Space complexity

The space complexity of this algorithm is $O(n)$ because we need to initialize a temporary array at least half the size of the array in order to perform the merge operation. In our case, to make implementation easier, we decided to use a temporary array that is the same size as the array, making exactly n , which is still $O(n)$.

Comparison with other sorting algorithms

Algorithm	Best Case	Average Case	Worst Case
Bubble Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$

²On the Worst-Case Complexity of TimSort by Nicolas Auger, Vincent Jugé, Cyril Nicaud, Carine Pivoteau is linked here

Algorithm	Best Case	Average Case	Worst Case
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n \log_2 n)$
Timsort	$O(n)$	$O(n \log_2 n)$	$O(n \log_2 n)$

It is important to note that Bubble sort, Selection sort can have a best case of $O(n^2)$ for sorted array if we implement early breaking mechanisms such as, if there has been no swaps since we've swept through the array, we break, although, the naive implementation (which is what we implemented) would run in $O(n^2)$ even for an already sorted array

Performance results and comparison with other algorithms

Note that the $\pm$ uncertainty range is in terms of the sample standard deviation σ . If we assume that it's normally distributed, we can estimate that 99.7% of the sample falls in between $\mu \pm 3\sigma$, where μ is the sample mean. If we remain conservative and assume that the data is of an arbitrary distribution, Chebyshev's theorem gives us that 99% of the samples must fall in between the range of $\mu \pm 10\sigma$.

Also note that we will sometimes refer to the array of sized 5 as small sized, the array of sized 1000 as medium sized, and the array of sized 1000000 as large sized.

The raw results are stored here on GitHub, while the analyzed results are stored here on GitHub.

Runtime

Note that the numbers are in seconds.

With $r = 0.1$:

Algorithm	5	1000	1000000
<code>bubbleSort</code>	$1.13 \times 10^{-5} \pm 7.90 \times 10^{-5}$	$2.31 \times 10^{-3} \pm 9.53 \times 10^{-4}$	N/A
<code>insertionSort</code>	$5.35 \times 10^{-6} \pm 7.69 \times 10^{-6}$	$8.43 \times 10^{-4} \pm 7.57 \times 10^{-4}$	N/A
<code>javaSort</code>	$4.09 \times 10^{-4} \pm 4.06 \times 10^{-3}$	$2.28 \times 10^{-4} \pm 8.95 \times 10^{-5}$	$2.68 \times 10^{-1} \pm 2.25 \times 10^{-2}$
<code>mergeSort</code>	$5.06 \times 10^{-6} \pm 8.06 \times 10^{-6}$	$6.32 \times 10^{-4} \pm 1.84 \times 10^{-4}$	$4.53 \times 10^{-1} \pm 1.41 \times 10^{-2}$
<code>selectionSort</code>	$3.90 \times 10^{-6} \pm 6.84 \times 10^{-6}$	$1.24 \times 10^{-3} \pm 8.31 \times 10^{-4}$	N/A
<code>timSort</code>	$1.11 \times 10^{-5} \pm 4.69 \times 10^{-5}$	$2.90 \times 10^{-4} \pm 9.38 \times 10^{-5}$	$4.55 \times 10^{-1} \pm 2.59 \times 10^{-2}$

With $r = 10$:

Algorithm	5	1000	1000000
bubbleSort	$3.29 \times 10^{-6} \pm 7.40 \times 10^{-6}$	$3.09 \times 10^{-3} \pm 1.01 \times 10^{-3}$	N/A
insertionSort	$3.24 \times 10^{-6} \pm 6.32 \times 10^{-6}$	$8.74 \times 10^{-4} \pm 6.65 \times 10^{-4}$	N/A
javaSort	$1.47 \times 10^{-6} \pm 4.62 \times 10^{-6}$	$9.27 \times 10^{-5} \pm 1.41 \times 10^{-5}$	$2.73 \times 10^{-1} \pm 2.41 \times 10^{-2}$
mergeSort	$4.87 \times 10^{-7} \pm 9.35 \times 10^{-8}$	$1.70 \times 10^{-4} \pm 1.24 \times 10^{-5}$	$4.50 \times 10^{-1} \pm 2.07 \times 10^{-2}$
selectionSort	$4.73 \times 10^{-6} \pm 1.13 \times 10^{-5}$	$1.31 \times 10^{-3} \pm 8.29 \times 10^{-4}$	N/A
timSort	$2.45 \times 10^{-6} \pm 4.37 \times 10^{-6}$	$1.69 \times 10^{-4} \pm 1.82 \times 10^{-5}$	$4.39 \times 10^{-1} \pm 1.03 \times 10^{-2}$

The reason why some of the values for the 1000000 column is missing is because, those are $O(n^2)$ sorting algorithms, as such, they wouldn't be able to finish in a reasonable amount of time.

This can easily be seen with some back-of-the-envelope calculation.

When we move from an input size of 1000 to 1000000, we're increasing the input size by a factor of 1000. As such, the amount of time it would take would be scaled by a factor of $1000^2 = 10^6$ or a million.

Since it took $2.31 \times 10^{-3}s$ for Bubble Sort to sort a sized 1000 array, for a sized 10^6 array, we would expect the algorithm to take take:

$$2.31 \times 10^{-3} \times 10^6 = 2.31 \times 10^3 = 2310s$$

It would then take around 38.5 minutes to complete. If we were to run this 100 times, it would then take around 3850 minutes to complete, which is around 64.2 hours. As such, for $O(n^2)$ sorting algorithms, which in our case, include Bubble Sort, Selection Sort, and Insertion Sort; they won't have an entry in the array of sized 1000000 column of any of the other metrics either.

For data with low duplication, Timsort and Merge Sort performs very similarly for large input (10^6 length array), although, it's about 2 times slower than Java's implementation of `Arrays.sort`. This is somewhat expected since for data with low duplication, most of the runs will be of size `minRun`, which makes it behave similarly to Merge Sort.

Where Timsort runs somewhat faster than Merge Sort in the case of data with low duplication with sized 1000, although, it might not be a statistically significant difference given the uncertainty.

As for the data with high duplication, the results doesn't seem significantly different, perhaps because the duplication ratio is too low.

All in all, for our particular dataset, Timsort performs very similarly to Merge Sort, and since Timsort is $O(n \log_2 n)$, it can sort an array of sized 1000000 in a reasonable amount of time compared to Bubble Sort, Selection Sort, and Insertion Sort.

Comparisons

With $r = 0.1$:

Algorithm	5	1000	1000000
bubbleSort	10 ± 0	$4.995 \times 10^5 \pm 0$	N/A
insertionSort	13.47 ± 3.00	$1.906 \times 10^4 \pm 44.92$	N/A
javaSort	N/A	N/A	N/A
mergeSort	7.18 ± 0.85	$8.709 \times 10^3 \pm 16.35$	$1.867 \times 10^7 \pm 555.35$
selectionSort	14 ± 0	$5.005 \times 10^5 \pm 0$	N/A
timSort	16.35 ± 2.96	$1.342 \times 10^4 \pm 67.41$	$2.343 \times 10^7 \pm 1.627 \times 10^3$

With $r = 10$:

Algorithm	5	1000	1000000
bubbleSort	10 ± 0	$4.995 \times 10^5 \pm 0$	N/A
insertionSort	4 ± 0	$1.894 \times 10^4 \pm 71.26$	N/A
javaSort	N/A	N/A	N/A
mergeSort	7 ± 0	$8.701 \times 10^3 \pm 16.12$	$1.867 \times 10^7 \pm 611.02$
selectionSort	14 ± 0	$5.005 \times 10^5 \pm 0$	N/A
timSort	4 ± 0	$1.337 \times 10^4 \pm 75.44$	$2.343 \times 10^7 \pm 2.075 \times 10^3$

Note that for Java's sort, we couldn't count the amount of swaps, nor the amount of comparisons because they don't use our particular implementation of swap and comparison which has innate metrics logging.

The amount of comparisons has a standard deviation that's quite tight, which is good for determining the difference between different algorithms.

For the medium sized array, Timsort has fewer comparisons than Insertion Sort, however, Merge Sort has a significantly lower amount of comparisons. This is probably due to the fact that Timsort requires comparisons for merging similar to Merge Sort, but also comparisons to determine the amount of runs, whether it's decreasing, and because we use Insertion Sort for smaller sized array.

Swaps

With $r = 0.1$:

Algorithm	5	1000	1000000
bubbleSort	4.92 ± 1.98	$2.492 \times 10^5 \pm 5.133 \times 10^3$	N/A
insertionSort	0 ± 0	0 ± 0	N/A
javaSort	N/A	N/A	N/A
mergeSort	0 ± 0	0 ± 0	0 ± 0
selectionSort	2.97 ± 0.66	992.43 ± 2.36	N/A
timSort	1.14 ± 1.00	246.72 ± 42.51	$2.500 \times 10^5 \pm 1.366 \times 10^3$

Algorithm	5	1000	1000000
-----------	---	------	---------

With $r = 10$:

Algorithm	5	1000	1000000
bubbleSort	0 ± 0	$2.467 \times 10^5 \pm 4.822 \times 10^3$	N/A
insertionSort	0 ± 0	0 ± 0	N/A
javaSort	N/A	N/A	N/A
mergeSort	0 ± 0	0 ± 0	0 ± 0
selectionSort	3 ± 0	993.42 ± 2.21	N/A
timSort	0 ± 0	247.52 ± 44.52	$2.499 \times 10^5 \pm 1.244 \times 10^3$

Bubble Sort and Selection Sort requires swaps, however, Insertion Sort and Merge Sort doesn't do a conventional swap, as such, this isn't quite a fair metric of comparison.

Copies (excluding swaps)

With $r = 0.1$:

Algorithm	5	1000	1000000
bubbleSort	0 ± 0	0 ± 0	N/A
insertionSort	11.04 ± 3.38	$2.517 \times 10^5 \pm 4.861 \times 10^3$	N/A
javaSort	N/A	N/A	N/A
mergeSort	24 ± 0	$1.995 \times 10^4 \pm 0$	$3.990 \times 10^7 \pm 0$
selectionSort	0 ± 0	0 ± 0	N/A
timSort	8.03 ± 2.91	$1.940 \times 10^4 \pm 173.41$	$3.941 \times 10^7 \pm 5.546 \times 10^3$

With $r = 10$:

Algorithm	5	1000	1000000
bubbleSort	0 ± 0	0 ± 0	N/A
insertionSort	0 ± 0	$2.500 \times 10^5 \pm 4.928 \times 10^3$	N/A
javaSort	N/A	N/A	N/A
mergeSort	24 ± 0	$1.995 \times 10^4 \pm 0$	$3.990 \times 10^7 \pm 0$
selectionSort	0 ± 0	0 ± 0	N/A
timSort	0 ± 0	$1.929 \times 10^4 \pm 162.64$	$3.941 \times 10^7 \pm 5.402 \times 10^3$

The copies excluding swaps gets counted when we do things such as right shift operators and merging. Nonetheless, it still isn't a fair metric of comparison because, Bubble Sort and Selection Sort only uses swaps, which doesn't get counted into the copies excluding swaps.

Copies (including swaps)

With $r = 0.1$:

Algorithm	5	1000	1000000
bubbleSort	14.76 ± 5.95	$7.477 \times 10^5 \pm 1.540 \times 10^4$	N/A
insertionSort	11.04 ± 3.38	$2.517 \times 10^5 \pm 4.861 \times 10^3$	N/A
javaSort	N/A	N/A	N/A
mergeSort	24 ± 0	$1.995 \times 10^4 \pm 0$	$3.990 \times 10^7 \pm 0$
selectionSort	8.91 ± 1.98	$2.977 \times 10^3 \pm 7.07$	N/A
timSort	11.45 ± 4.47	$2.014 \times 10^4 \pm 224.68$	$4.016 \times 10^7 \pm 6.637 \times 10^3$

With $r = 10$:

Algorithm	5	1000	1000000
bubbleSort	0 ± 0	$7.402 \times 10^5 \pm 1.447 \times 10^4$	N/A
insertionSort	0 ± 0	$2.500 \times 10^5 \pm 4.928 \times 10^3$	N/A
javaSort	N/A	N/A	N/A
mergeSort	24 ± 0	$1.995 \times 10^4 \pm 0$	$3.990 \times 10^7 \pm 0$
selectionSort	9 ± 0	$2.980 \times 10^3 \pm 6.64$	N/A
timSort	0 ± 0	$2.004 \times 10^4 \pm 205.08$	$4.016 \times 10^7 \pm 6.658 \times 10^3$

The copies which include swaps includes swaps into the total amount of copies, where one swap amounts to 3 copies operations.

For arrays with a high low duplication factor, Bubble Sort, and Timsort doesn't do any copy and exits for small arrays. The array consists of just 1, meaning that it is already sorted. Merge Sort on the other hand is naive, and still splits the array, even though it's already sorted. Similarly for Selection Sort, it will still methodically search for the minimum and try to swap it, because it's naive.

For bigger arrays, for both arrays with low and high duplication factor, the amount of copies between Merge Sort and Timsort isn't significant. We believe that Timsort would perform a lot better for data more duplicates, and larger runs (natural runs larger than 32).

Use Cases & Real-World Examples

Since Timsort is a hybrid architecture makes it highly advantageous for specific software, although its memory requirements limit its universal application.

Optimal Scenarios (Appropriate Use Cases):

- Real world Data Processing: in this case data often contains pre-existing sequential patterns (partially sorted data); Timsort capitalizes on these natural runs to approach its best case which is the $O(n)$.
- Stability Critical Tasks: Since Timsort is a stable sorting algorithm (assuming that merging is implemented in a stable fashion, which it can be), it is ideal for sorting complex objects in arrays by multiple fields sequentially without disrupting the original relative order of equal elements.
- High-Level Language Core Libraries: Timsort is well optimized for the standard library implementations handling drivers, with unpredictable user datasets, as evidenced by its integration into Python's `list.sort` method (pre-Python 3.11) , Java's `Arrays.sort` ³.

Unoptimal Scenarios (Inappropriate Use Cases):

- Limited Ram: Timsort require auxiliary memory proportional to the data size, it is inappropriate for embedded system that has only the $O(1)$ space requirement.
- Integer Arrays: For Integer arrays, sorting can be done a lot faster using an algorithm like Radix Sort. We choose an array of Integers for testing due to its simplicity.

Conclusion and Limitations

Timsort is seen as the gold standard in terms of general sorting algorithm. Although this demonstration haven't fully demonstrated Timsort's capability for real world data, we've shown that for random data, it performs just as good as Merge sort, which has an $O(n \log_2 n)$ run time, making it feasible for arrays of very large sizes.

In practice, Timsort also has many more optimizations such as galloping mode for merging, and more advanced techniques which we haven't implemented which may make it a lot faster. In the future, it would be a lot more insightful to use datasets that more resemble real world data, and implement the different optimizations that is used in applications such as in Java's `Arrays.sort`.

³Python moved away from using Timsort to using Power Sort starting from Python version 3.11. More information can be found here.